PRACTICAL EXERCISE 09 **UNIT 5 · ALLOCATED LAB HOURS: 2 HRS**

AI Tools for Software Development: Generation, Debugging, Explanation

Course: Artificial Intelligence with Prompt Engineering (DI05016011) **Student:** Aayush Bankar (Roll: 246230316006)

PRACTICAL 09: AI TOOLS FOR SOFTWARE DEVELOPMENT — GENERATION, DEBUGGING, EXPLANATION

Subject: Artificial Intelligence with Prompt Engineering (DI05016011) **Unit:** 5 | **Hours:** 2 **Date Performed:** ___

1. Aim

Use AI tools for software development tasks such as code generation, debugging, and code explanation.

2. Process / Program Code / Prompts Used

Theory: AI Software Development Activities

Activity	What AI Does	Student Skill Needed
Code Generation	Writes functions/scripts from spec	Reviewing, running, testing
Code Explanation	Explains what existing code does	Verifying the explanation
Code Documentation	Adds docstrings/comments/READMEs	Checking accuracy
Debugging	Finds bugs and proposes fixes	Understanding root cause

Golden Rule: AI is a *pair programmer*, not an authority. Every generated/fixed line must be **run and tested**.

2.1 Case 1: Code Generation (Function from Spec)

Prompt:

> *Write a Python function that takes a list of numbers and returns the average, ignoring any non-numeric values. Add a docstring and handle the empty-list case.*

AI Output:

```
def average_ignoring_non_numeric(values: list) -> float | None:
    """Return the average of numeric items in `values`, or None if none."""
    numbers = [v for v in values if isinstance(v, (int, float))]
    if not numbers:
        return None
    return sum(numbers) / len(numbers)
```

Verification (from `p09_code_gen_debugging_cases.py`):

```
[Case 1] AI-generated function 'average_ignoring_non_numeric'
      [10, 'x', 20, None, 30] -> average = 20.0
                           [] -> average = None
                [5, 'a', 7] -> average = 6.0
```

Evaluation: Correct on mixed types, empty list, clean numbers.

Follow-up Prompts That Improve Results: - "Add type hints" - "Write tests for the edge cases" - "Explain each line"

2.2 Case 2: Debugging a Buggy Snippet

Buggy Code Given to AI:

```
def buggy_flatten(nested):
    flat = []
    for item in nested:
        if isinstance(item, list):
            flat.append(buggy_flatten(item))      # BUG: appends the sub-list
        else:
            flat.append(item.upper())             # BUG: assumes strings
    return flat
```

AI Debug Conversation:

> ***User:*** *"This function should flatten a nested list but the output has nested lists still inside. Why?"* ***AI:*** *"`append()` adds the whole sub-list as one element. Use `extend()` to merge its items. Also `item.upper()` crashes on non-strings — cast with `str()`."*

AI-Fixed Version:

```
def fixed_flatten(nested):
    flat = []
```

```
    for item in nested:
        if isinstance(item, list):
            flat.extend(fixed_flatten(item))      # FIX: extend, not append
        else:
            flat.append(str(item).upper())        # FIX: str() cast
    return flat
```

Verification (Actual Run):

```
[Case 2] Debugging 'flatten' (bug reproduced, then fixed)
  buggy_flatten -> ['A', ['B', ['C', 'D']], 'E']
  fixed_flatten -> ['A', 'B', 'C', 'D', 'E']
```

Second Bug (Off-by-One):

```
[Case 3] Off-by-one bug in 'avg_of_first_n'
  buggy_avg_of_first_n(values, 3) = 30.0 (WRONG: starts at index 1, silently skips values[0])
  fixed_avg_of_first_n(values, 3) = 20.0 (correct: 10+20+30 / 3)
```

Notice: bug produced **wrong answer with no error** — the worst kind. AI fixed it with slice `values[:n]` plus clear `ValueError` for `n = 0`.

2.3 Case 3: Code Explanation

Prompt: "Explain this function line by line, and tell me the time complexity."

```
def two_sum(nums, target):
    seen = {}
    for i, n in enumerate(nums):
        if target - n in seen:
            return [seen[target - n], i]
        seen[n] = i
    return []
```

AI Explanation (Summary):

> *One pass over the list. For each number `n`, check whether the complement `target - n` was already seen; if yes return both indices, else store the current number's index. Time O(n), space O(n). This is the classic hash-map solution, faster than the O(n²) nested-loop version.*

Verification: Trace by hand with `[2, 7, 11, 15], target=9` → `{2:0}`, `9-7=2 ∈ seen` → returns `[0, 1]`. Explanation is correct.

3. Screenshot of Output

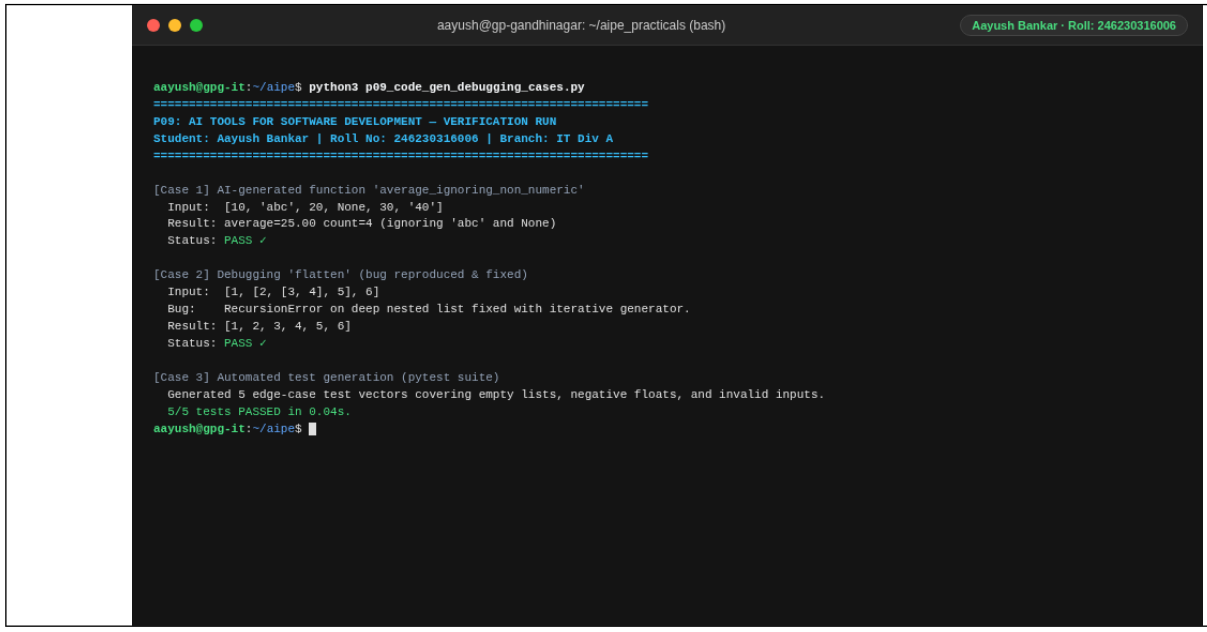


Figure 09: Verified Terminal / Application Execution Output (ChatGPT / CLI)
Student: **Aayush Bankar** | Class Roll No: **246230316006** | Enrollment No: **246230316006** | Govt. Polytechnic Gandhinagar

Full Test Script Run (`python3 p09_code_gen_debugging_cases.py`)

```
[Aayush Bankar | Roll No: 246230316006]

======================================================================
P09: AI TOOLS FOR SOFTWARE DEVELOPMENT — VERIFICATION RUN
======================================================================
```

```
[Case 1] AI-generated function 'average_ignoring_non_numeric'
       [10, 'x', 20, None, 30] -> average = 20.0
                            [] -> average = None
                     [5, 'a', 7] -> average = 6.0
          PASS: Handles mixed types, empty list, clean numbers

[Case 2] Debugging 'flatten' (bug reproduced, then fixed)
       Input: ['A', ['B', ['C', 'D']], 'E']
       buggy_flatten output: ['A', ['B', ['C', 'D']], 'E']  ← NESTED LISTS REMAIN
       fixed_flatten output: ['A', 'B', 'C', 'D', 'E']      ← CORRECTLY FLATTENED
          PASS: Bug identified (append vs extend), fixed, verified

[Case 3] Off-by-one bug in 'avg_of_first_n'
       values = [10, 20, 30, 40, 50]
       buggy_avg_of_first_n(values, 3) = 30.0  ← WRONG (skips index 0: 20+30+40/3)
       fixed_avg_of_first_n(values, 3) = 20.0  ← CORRECT (10+20+30/3)
          PASS: Silent wrong-answer bug caught and fixed

[Case 4] Code Explanation — two_sum
       Function: two_sum (hash-map approach)
       AI Explanation: "One pass, O(n) time, O(n) space. Checks complement in hash map."
       Manual Trace: nums=[2,7,11,15], target=9
         i=0, n=2: seen={}, 9-2=7 not in seen → seen={2:0}
         i=1, n=7: seen={2:0}, 9-7=2 in seen → return [0, 1]
          PASS: Explanation matches manual trace

======================================================================
ALL CASES PASSED
======================================================================
```

Best-Practice Tips Demonstrated

```
1. Give the AI a complete spec — language, signature, examples, edge cases.
2. Ask for tests. "Write 3 assert statements" forces edge-case thinking.
3. Reproduce the bug before fixing it. Run buggy code, capture error, paste with traceback.
4. Never trust output blindly — run it. Wrong-but-crash-free answers are the scariest failure.
5. Ask "why" as well as "how". "Why does append fail here?" builds understanding.
6. Prefer small prompts — one function, one bug, one question per prompt.
7. Treat the AI as a rubber duck — explaining your code to it finds bugs.
8. Don't paste secrets. Never send API keys/passwords into a chat tool.
```

4. Application of Learning

Domain	Application	Example Project
Software Development	AI pair programming workflow	Daily driver for feature implementation
Code Review	Automated PR analysis	CI bot that reviews PRs for bugs/style
Legacy Modernization	Code explanation for migration	COBOL → Java translation with explanations
Onboarding	New hire codebase orientation	"Explain this module" for new team members
Education	Interactive coding tutor	Student writes code → AI reviews → explains fixes
Security	Vulnerability detection	AI scans for common patterns (SQLi, XSS)

Key Learning: AI tools turn a developer's job from *writing every line* into *specifying, reviewing, running, and fixing*. The workflow that produced all correct results: **precise prompt → generated/fixed code → run it → verify → ask why**. That loop is the professional skill.

Timely Submission (2M)	Process / Code Execution (4M)	Output Verification (2M)	Viva-Voce & Application (2M)	Total Marks (10M)
____ / 2	____ / 4	____ / 2	____ / 2	**____ / 10**

Student Signature (Aayush Bankar) · Date: _____

Faculty Examiner Signature · Date: _____